CodeQuotient

Department of Computer Science & Engineering

LAB MANUAL

22CS025 - Algorithm Design & Implementation-5Sem-2022

Student Name	**JOEL MATTHEW**
Email	**joel465.be22@chitkara.edu.in**
Course Name	**22CS025 - Algorithm Design & Implementation-5Sem-2022**

Faculty Incharge

_____

Aim: 0-1 Knapsack problem

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.
In this problem we will solve a variant of it, in which the items can have a binary decision only i.e. either you can pick the item or leave it. No partial items allowed to fill the bag at fullest. This is also known as 0-1 knapsack problem.
Given two integer arrays values[0..n-1] and weight[0..n-1] which represent values and weights associated with n items respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of values[] such that sum of the weights of this subset is smaller than or equal to W.
Note: You cannot break an item, either pick the complete item, or don't pick it (0-1 property).
Input Format
First Line will contain an integer N denoting the number of items.
Second line contains N integers separated by space as values of N items.
Third line contains N integers separated by space as weights of N items.
Fourth line contains an integer denoting the capcacity of knapsack.
Output Format
Print the maximum value that can be earned with above knapsack.
Constraints
1 <= N <= 10^3
1 <= val[i] <= 10^3
1 <= weight[i] <= 10^3
1 <= capacity <= 10^3
Sample Input
3
60 100 120
10 20 30
50
Sample Output
220
Explanation
Pick 2nd and 3rd item with weight 20 and 30 and profit 100 and 120.

Solution:

```
class Result
{
   static int zeroOneKnapsack(int val[], int weight[], int n, int capacity) {
      int[][] dp = new int[n + 1][capacity + 1];
      for (int i = 1; i <= n; i++) {
         for (int j = 1; j <= capacity; j++) {
            dp[i][j] = weight[i - 1] <= j
                  ? Math.max(dp[i - 1][j], dp[i - 1][j - weight[i - 1]] + val[i - 1])
                  : dp[i - 1][j];
         }
      }
      return dp[n][capacity];
   }
```


JOEL MATTHEW
joel465.be22@chitkara.edu.in

```
}
```


JOEL MATTHEW
joel465.be22@chitkara.edu.in